

DL050 RNN 与 BPTT 实验指导

这是一份可独立使用的实验单。先完成纸笔与代码检查，再看 answers.md。你将从两条短序列的完整数值链出发，验证反向传播，再运行一个延迟符号任务，并写出有边界的研究结论。任务不需要联网、不需要 GPU、不下载任何外部数据。

1 目标与交付内容

你应提交一份记录，包含：逐样本逐时间前向与反向表；共享参数梯度和同步更新后的下一次前向；有限差分误差；完整 27 组实验的结果而非最好 seed；一个明确标为事后的诊断；以及三条“当前结果尚不能证明”的陈述。无需追求把所有测试准确率调到 1，保留失败与负面结果是实验的一部分。

先修是 DL027、DL035、DL049。需要能读 NumPy 数组、矩阵乘法、mean 分母、tanh 导数和序列 mask。正文复习了这些前提。若不熟悉矩阵反传，先完成 $D=H=1$ 的任务，不要跳到训练曲线。

2 文件和环境

lecture.md 与 lecture.pdf 是主讲义；本文件和 lab.pdf 是实验步骤；answers.md 与 answers.pdf 提供详细答案。experiment.ipynb 包含真实执行的完整流程与 PNG。reference.py 为独立 NumPy 参考，experiment.py 为实际 PyTorch 训练入口，test_experiment.py 是可在正常与 -O 模式下运行的行为检查。

数据已经保存在 data/delayed_sign.npz，可用 generate_data.py 原样重建。protocol.json 固定数据 seed、模型 seed、超参、终点与汇报规则。outputs/results.json 保存每一步的训练损失与梯度范数，trained_parameters.npz 保存每一组的终点参数，hand_calculation.json 保存完整手算账本。

建议 Python 3.12 的隔离环境，CPU PyTorch 2.7.1、NumPy 2.3.5。Linux 上可使用：

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
mkdir -p .work/tmp .work/mpl .work/cache
export TMPDIR="$PWD/.work/tmp"
export MPLCONFIGDIR="$PWD/.work/mpl"
export XDG_CACHE_HOME="$PWD/.work/cache"
```

requirements.txt 中 torch 的公开版本标记为 2.7.1；Linux CPU wheel 实际可显示 2.7.1+cpu。当前包不测试 CUDA、Apple MPS、float32 或混合精度。若只阅读已经生成的文件，不需要安装环境。PDF 构建额外依赖与字体见 build_tools/README.md。

在本单元目录运行：

```
python generate_data.py
python test_experiment.py
python -O test_experiment.py
python experiment.py --out .work/reproduction
```

最后一个命令会运行全部 27 组训练，不要在结束前用部分输出下结论。典型 CPU 单线程运行需数十秒到数分钟，实际取决于机器；内存应留至少 1 GB。本任务的规模远低于 API 支持的上限，禁止用资源上限当成可随意叠加的性能承诺。

读取已有原始结果只需打开 `outputs/results.json`，不会触发训练。若要运行 Notebook，安装 `notebook` 并打开 `experiment.ipynb` 后重启内核并运行全部。已提供的执行记录使用全新实际 IPython 进程内核；浏览器界面和 socket 传输未测试，不把内核执行等同于界面兼容性保证。

3 任务 A 先写清计算对象

1. 画出两条独立序列 A 与 B：A 输入 $[1, 0, -1]$ ，标签 $[1, 0, 1]$ ；B 输入 $[-1, 0.5]$ ，标签 $[0, 1]$ 。将 B 补齐到 3 步，标出长度和状态、损失两个 mask。
2. 固定 $U=0.4$ 、 $W=0.6$ 、 $b=0.1$ 、 $v=0.7$ 、 $c=-0.2$ ， $hA_0=hB_0=0$ 。说明哪些量是参数、哪些是状态、哪些是数据。
3. 写出逐有效位置平均损失的分母。若改成每条序列先平均再对两条序列平均，A1 和 B1 的权重各是多少？
4. 把标量写成 $D=2$ 、 $H=3$ 、 $N=4$ 、 $T=5$ 的矩阵形状表。解释为什么代码 $X[:,t] @ U.T$ 等于数学列向量 Ux 。
5. 说明本讲连续输入怎样衔接 DL049 的 $E[\text{token}]$ ，以及为什么不能对离散 token ID 求普通实数导数。

交付：一张图、一张形状表、两个归约目标。不允许只写“使用 mask”，要说明 mask 作用在哪个操作、哪个分母。

4 任务 B 完整纸笔数值链

先独立填写以下字段，至少保留 8 位中间有效数字，最终显示 6 位即可。用计算器算 $\tanh$ 与 sigmoid 是允许的；不要读自动生成的答案表后再填写。

前向表每个有效位置填写 x 、 $h_{\text{前}}$ 、 a 、 h 、 z 、 p 、 y 、单位位置 BCE，最后相加除以有效位置数。B3 说明为何没有损失；它的采用状态是什么？

反向从各序列末尾开始，每个有效位置填写 $e=(p-y)/Z$ 、 ve 、来自未来的 $W\delta$ 下一步、总状态梯度、 $\tanh$ 局部导数、 δ 。特别解释 A2 两条路径为什么符号相反，以及无直接损失的时刻是否必然无梯度。

参数表每个位置填写 δ_x 、 $\delta_{h_{\text{前}}}$ 、 δ 、 eh 、 e 五个贡献，沿样本与时间累加。写出 A1、B1 对 W 局部贡献为零的原因；写出 A2 对 U 局部贡献为零的原因，但不能据此跳过其反传。

用 $\eta=0.2$ 同步更新全部五个参数。使用新参数、重新置零的两个初态，完整重算下一次前向和平均损失。指出哪几个位置的损失反而增大，解释为何与总体改善并不矛盾。

交付：旧参数前向、完整反向、共享参数汇总、更新、新前向共五张表。只提交“loss 下降了”不算完成。

5 任务 C 推导与实现独立反传

在不调用 `autograd` 的前提下实现下列接口，然后再与 `reference.py` 对照：输入参数字典、 $X: [N,T,D]$ 、 $\text{lengths}: [N]$ 、目标 $Y: [N,T]$ ；输出 `loss`、全部 `h`、每个 δ 、各参数梯度、输入梯度及对 `h0` 的梯度。

先写有效位置公式，再写包含冻结状态的严格 `mask` 公式。逆序循环里要同时包含候选分支和恒等分支。常见错误是把 `m=0` 直接当作“所有上游梯度丢弃”，而不是沿采用的状态分支传播。

1. 对纸笔例比较 Python `math` 标量前向、NumPy 手写反传与 PyTorch。
2. 再用 $N=3$ 、 $T=4$ 、 $D=2$ 、 $H=3$ ，长度 $[4,2,1]$ 的小数组检查。解释为何标量例不足以发现转置或求和轴错误。
3. 对每个参数和每个输入标量做中心有限差分， ε 分别用 10^{-4} 、 10^{-5} 、 10^{-6} 。记录最大绝对误差及误差是否随 ε 改变。
4. 对接近零的梯度，同时看绝对误差和相对误差，说明不能只设置一个相对误差阈值。
5. 临时把三个时刻的 W 解绑定成 $W1$ 、 $W2$ 、 $W3$ ，保持数值相同。证明并测试三份梯度的和等于共享 W 梯度。

可采用归一化误差：

$$r = \frac{|g_{FD} - g|}{\max(10^{-8}, |g_{FD}| + |g|)}.$$

但报告仍需包括绝对误差。测试通过是对这些输入的证据，不是对任意长序列或浮点范围的证明。

6 任务 D 定位边界错误

依次实施以下干预，记录预测、损失与梯度是否应改变，并解释理由。

- 将 B 的 `padding` 输入改成 9，`padding` 标签从 0 改成 1，保持长度为 2
- 交换 `batch` 两个样本的顺序，再把结果交换回来
- 将每个样本分别前向，与同批前向比较
- 故意让 $B1$ 使用 $A3$ 的状态，计算错误结果
- 仅在损失处 `mask`，去掉状态冻结，再读补齐数组的最后时刻
- 在第 2 步之前 `detach`，但保持同样参数和输入，比较前向值与早期输入梯度

最后两项的具体读出与边界必须写清楚，否则“结果变或不变”无法唯一判断。恢复正确实现后再继续训练，不要把有意制造的 bug 保留到主结果里。

7 任务 E 理解 Jacobian 与裁剪

1. 推导 $J_t = \text{diag}(1 - h_t^2)W$ ，给出 $\partial h_4 / \partial h_1$ 的矩阵次序，并解释反传为何出现转置。
2. 计算 0.5^{20} 、 0.9^{20} 、 1.1^{20} 、 1.5^{20} 。它们代表人为固定标量 Jacobian 的机制示意，不是训练曲线。
3. 取 $W=1.5$ ，分别计算 $a=0$ 与 $a=3$ 时的 $|J|$ 。说明 $W>1$ 不能单独推出实际梯度爆炸。
4. 对 $g=(3,4)$ 、 $\tau=2$ ，计算全局范数裁剪与逐元素裁剪。检查方向和范数。
5. 解释为什么在各时间步贡献求和前分别裁剪会得到不同梯度。
6. 写一个测试，确认真实 step 的更新来自同一组旧参数；与 NumPy 末端损失参考比较下一次前向，而不仅比较当前 loss。

交付：完整式子、数值及一句成立条件。不要把上界、充分条件和双向充要条件混用。

8 任务 F 运行固定的延迟实验

先阅读 protocol.json，在运行前抄下假设：延迟增长可能增加训练困难；裁剪可能稳定部分运行；截断会删除部分信用分配路径，但不保证长任务一定失败。将“梯度正确性”和“任务表现”分开预测。

实验是首位 ± 1 符号记忆：delay=2、8、24；N训练=256、N验证=128、N测试=256；H=12；seeds=7、19、41；float64 CPU；固定 320 步；完整 batch SGD、lr=0.3。三组是 full、full_clip($\tau=0.25$)、tbptt4_clip($K=4, \tau=0.25$)。所有独立序列从零状态开始。

运行真实入口，不允许在本轮看到验证或测试结果后改参数。若提出新阈值或新结构，应另开明确标记的探索性实验，保持原始 27 组结果完整。保存运行环境、协议散列和全部 seed；失败同样保存错误信息。

逐组记录训练、验证、测试 BCE 和准确率；记录每步裁剪前后范数；保留所有训练曲线。汇总均值与样本 SD，同时展示原始点。确认 delay=2 时 full_clip 与 tbptt4_clip 完全相同，因为没有实际截断边界。

固定块 $K=4$ 在长度 $T=9$ 和 $T=25$ 时，最终块各有几步？最终损失可以回传至哪个输入？这是解释结果前必须回答的问题。

9 任务 G 给负面结果正确命名

阅读并回答：

1. 如果 delay=24 的 full 接近随机基线，而 full_clip 好很多，你能否断言裁剪总是优于不裁剪？还缺哪些对照？
2. 如果 tbptt4_clip 在 delay=24 做得很好，能否据此断言它使用了完整 24 步梯度？如何直接检查？

3. 若三组的 accuracy 都等于 1, 为什么还记录 BCE、输入敏感度和训练曲线?
4. 3 个 seed 的平均 $\pm$ SD 能否解释为总体 95% 置信区间? 为什么测试样本数不能简单乘 3?
5. 本实验保留全部状态以便展示, 能否凭 detach 存在就宣称显存节省? 应怎样测量真正的分块实现?
6. 多次重新计算固定测试量与利用测试集调参有什么区别? 哪种行为会破坏封存用途?

交付: 一段不超过 250 字的主要结论, 加上完整数据附表, 至少包含一个不符合最初直觉的结果。

10 任务 H 事后诊断与研究设计

使用 `diagnostics.py`, 在训练完成后做两个干预: 翻转首位信号与标签、擦除首位信号。保留干扰不变。声明这是事后诊断, 不是首次训练前预注册的主要指标, 不用其结果选择 seed 或模型。

再选择一个尚未运行的新问题写实验计划, 不必执行: 改变最终块的长度; 引入随机延迟以打破固定时间捷径; 比较无中间损失与有中间辅助损失; 或比较等训练时间的真正流式 TBPTT。计划应包含假设、控制变量、split、seed、选择规则、停止标准、负面结果保留和资源预算。

不能从本 toy 数据直接推导自然语言任务、长文理解或科研系统的真实可靠性。把这个限制写在结论附近, 不要只放在附录末尾。

11 完成标准与故障处理

必须同时满足: 形状与归约解释清楚; 数值链完整; 独立参考而非两个相同实现互相印证; 有限差分与生产一步校验可复验; 状态边界测试符合预期; 完整 27 组实验保留; 图例不遮挡数据; Notebook 实际执行; 结论与证据范围一致。

不支持的输入明确报错: 空序列、长度 0 或超过 T、非整数长度、float32、NaN/Inf、超出代码上限的维度或幅度、不合法的截断步数。遇到报错先检查输入域, 不应删除验证代码来“使实验跑通”。

如果 PDF 字体显示方框, 按 `build_tools/README.md` 安装官方 Noto CJK 字体并重新生成图, 不能把缺字图用于证据; 如果 Jupyter 无法启动但命令行检查能跑, 记录界面/传输问题, 不把它误诊为模型数学错误。若出现数值分歧, 先对比环境版本、dtype、protocol SHA256、数据数组散列、状态重置和更新时机。